Graphs II: Shortest Paths

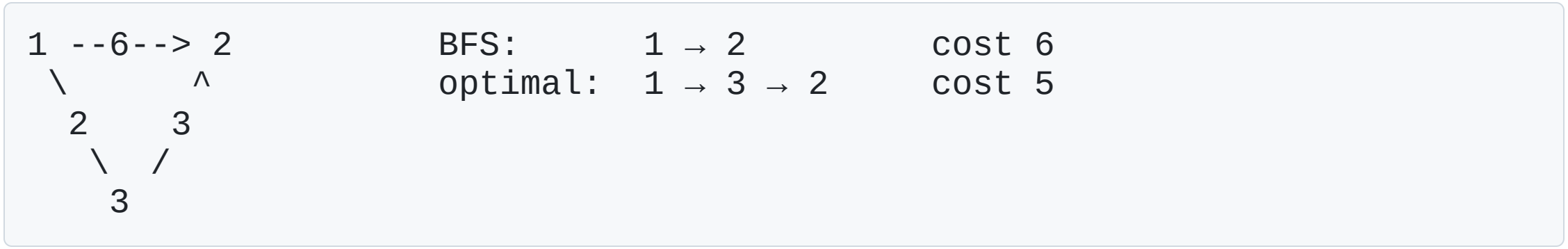
Dijkstra, Bellman-Ford, Floyd-Warshall and 0-1 BFS

Learning goals

- Implement **Dijkstra** with a priority queue and lazy deletion
- Explain **why** Dijkstra breaks with negative edges
- Use **Bellman-Ford** for negative weights and to **find negative cycles**
- Use **Floyd-Warshall** for all-pairs distances on small graphs
- Use **0-1 BFS** with a deque when weights are only 0 and 1
- Choose the right algorithm from n, m and the weight range

From BFS to weighted graphs

BFS (Week 16) minimises the **number of edges**. With weights, fewer edges ≠ cheaper:



Need algorithms that minimise the **sum of weights**.

The key operation: relaxation

`dist[v]` = best distance found so far (∞ initially, `dist[s] = 0`)

```
relax(u → v, w):
    if dist[u] + w < dist[v]:
        dist[v] = dist[u] + w
        par[v]  = u
```

All three single-source algorithms are "relax edges in a clever order until nothing changes". They differ only in the **order**.

Dijkstra: idea

Weights **non-negative**. Grow a set of "finished" vertices.

1. Pick the unfinished vertex `u` with the smallest `dist[u]`
2. Mark it finished — its distance is now final
3. Relax all edges out of `u`

Why final? Any other path to `u` leaves the finished set through some vertex `x` with `dist[x] ≥ dist[u]`, then adds non-negative weights. So it can't be shorter.

Dijkstra trace

Edges: 1→2 (6), 1→3 (2), 3→2 (3), 1→3 (4) (CSES 1671 sample)

step	pop (d, u)	dist[1]	dist[2]	dist[3]
init	—	0	∞	∞
1	(0, 1)	0	6	2
2	(2, 3)	0	**5**	2
3	(5, 2)	0	5	2
4	(6, 2) stale → skip			

Output: `0 5 2`

Dijkstra code (lazy deletion)

```
vector<ll> dist(n + 1, INF);
priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;
dist[s] = 0; pq.push({0, s});
while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d != dist[u]) continue;              // stale entry
    for (auto [v, w] : adj[u])
        if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            pq.push({dist[v], v});
        }
}
```

O((n + m) log m). `greater<>` → min-heap. `long long` distances!

Why the stale check matters

- A vertex can be pushed many times (each improvement pushes a copy)
- Without `if (d != dist[u]) continue;` we re-scan its edges for every copy
- Worst case: O(m) copies × O(deg) scans → **TLE** on dense / adversarial tests

Alternative: `vector<bool> done`, skip if `done[u]`, set `done[u] = true` after popping.

Dijkstra fails with negative edges

```
1 --2--> 2 --(-5)--> 3          1 --1--> 3
```

- Pops 1, then 3 (dist 1) → declares dist[3] = 1 **final**
- Later finds 1 → 2 → 3 = 2 − 5 = −3 (but 3 was already "done")
- The "finished = final" argument used `w ≥ 0`
- The lazy version may re-push and fix it, but can take **exponential** time

Never patch Dijkstra for negative edges in contest — use Bellman-Ford (or SPFA with care).

Bellman-Ford

Relax **every** edge, n − 1 times.

```
dist[s] = 0;
for (int round = 1; round <= n - 1; round++)
    for (auto [a, b, w] : edges)
        if (dist[a] != INF && dist[a] + w < dist[b])
            dist[b] = dist[a] + w;
```

- After round k, `dist[v]` ≤ shortest path using ≤ k edges
- A shortest path (no negative cycle) has ≤ n − 1 edges → done
- O(n·m). Stop early if a round changes nothing.

Negative cycles

If round **n** still relaxes something → a negative cycle is reachable.

Finding the cycle (CSES 1197 Cycle Finding):

1. Let `x` = a vertex relaxed in round n
2. Do `x = par[x]` n times → now `x` is surely **on** the cycle
3. Follow `par` from `x` until you return to `x`; reverse

Init `dist = 0` for all vertices = virtual source to everyone → finds a cycle **anywhere**.

Negative cycles and "−∞" distances

CSES 1673 High Score: maximise score ⇒ negate weights, shortest path.

- A negative cycle matters only if it is **reachable from s** and **can reach t**
- Mark vertices relaxed in round n; check reachability (BFS both ways)
- If any marked vertex is on an s → t route → answer is −1 (unbounded)

Don't print "−∞" just because *some* negative cycle exists.

Floyd-Warshall: all pairs

`d[i][j]` = shortest i → j distance using only intermediate vertices from {1..k}

```
for (int k = 1; k <= n; k++)          // k OUTERMOST
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
```

- Init: `d[i][i] = 0`, `d[a][b] = min(d[a][b], w)` (parallel edges!), else ∞
- $O(n^3)$: $n \le$ ~500 comfortable
- Handles negative edges; negative cycle ⇔ some `d[i][i] < 0`

Floyd-Warshall: why k outside?

It's a DP over k (Week 15 style!):

```
d_k[i][j] = min( d_{k-1}[i][j],  d_{k-1}[i][k] + d_{k-1}[k][j] )
```

- Rolling the k dimension is safe: `d[i][k]` and `d[k][j]` don't change in round k
- Putting k inside → computes something else, **wrong answers**
- Bonus uses: transitive closure (`|=` / `&`), minimax paths, counting reachability

0-1 BFS

All weights are **0 or 1**? Use a **deque** instead of a heap.

```
dist[s] = 0; dq.push_back(s);
while (!dq.empty()) {
    int u = dq.front(); dq.pop_front();
    for (auto [v, w] : adj[u])
        if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            if (w == 0) dq.push_front(v);
            else        dq.push_back(v);
        }
}
```

Deque holds at most two distance values (d, d+1) in order → like BFS layers. **O(n + m)**.

0-1 BFS example: reversing edges

CodeChef REVERSE: directed graph; minimum edges to reverse so 1 reaches n.

Modelling trick: for each edge u → v add

- u → v with weight **0** (use as is)
- v → u with weight **1** (use reversed, pay 1)

Answer = 0-1 BFS distance from 1 to n.

Grid version: walking costs 0, breaking a wall costs 1 (AtCoder ABC 213 E).

Path reconstruction

Same as BFS: store `par[v] = u` whenever `dist[v]` improves.

```
vector<int> path;
for (int v = t; v != s; v = par[v]) path.push_back(v);
path.push_back(s);
reverse(path.begin(), path.end());
```

Only valid if `dist[t] < INF`. Floyd-Warshall: store `nxt[i][j]` instead.

Choosing an algorithm

Situation	Algorithm	Time
unweighted	BFS	$O(n + m)$
weights $\in \{0, 1\}$	0-1 BFS	$O(n + m)$
weights ≥ 0	Dijkstra	$O(m \log m)$
negative weights, single source	Bellman-Ford	$O(n \cdot m)$
negative cycle detection	Bellman-Ford (n-th round)	$O(n \cdot m)$
all pairs, $n \leq 500$	Floyd-Warshall	$O(n^3)$
DAG, any weights	DP in topological order	$O(n + m)$ (Week 21)

Modelling: Dijkstra on states

Vertices don't have to be cities. State = (city, extra info):

- CSES 1195 Flight Discount: state (city, coupon used?) → 2n vertices
- Fuel / tickets / time-of-day: (city, remaining resource)
- "k-th shortest", "number of shortest paths": keep extra arrays (CSES 1202 Investigation)

Rule: if the extra info has small range, multiply the graph.

Common pitfalls

- `int` distances overflow: 10^5 edges × 10^9 weight → `long long`
- `INF + w` overflow: use `INF = LLONG_MAX / 4` or check `dist[u] != INF`
- Max-heap by accident: forgot `greater<>` (or push `-d`)
- No stale-entry check → TLE
- Floyd-Warshall with `k` not outermost
- Floyd-Warshall: forgetting `min` with parallel edges, or `d[i][i] = 0`
- Undirected edge = **two** directed edges

Practice problems

- CSES 1671 Shortest Routes I · CSES 1672 Shortest Routes II
- CSES 1195 Flight Discount · CSES 1202 Investigation
- CSES 1673 High Score · CSES 1197 Cycle Finding
- CF 20C Dijkstra? (print the path)
- AtCoder ABC 213 E Stronger Takahashi (0-1 BFS)
- CodeChef REVERSE · CF 1063B Labyrinth

Summary

- Relaxation is the core; algorithms differ in relaxation order
- **Dijkstra**: non-negative weights, priority queue, skip stale entries
- **Bellman-Ford**: $n - 1$ rounds; round n detects negative cycles
- **Floyd-Warshall**: $O(n^3)$ DP over intermediate vertex k
- **0-1 BFS**: deque, front for 0, back for 1
- Model extra constraints as **state graphs**
- Next week: mid-year review and mock contest